

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для фитнес-центра с системой
онлайн-записи на тренировки, управлением расписанием и
личным кабинетом пользователя»

Выполнил: Минеева Анастасия Игоревна

Группа: 241-3211

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«___» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Минеевой Анастасии Игоревне,

обучающейся группы 241-3211,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для фитнес-центра с системой онлайн-записи на тренировки, управлением расписанием и личным кабинетом пользователя»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.
2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих программных продуктов по теме работы	10	22.03.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	26.03.2026	
Задача 1.3. Формулировка цели и задач работы	8	30.03.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории	5	02.04.2026	
Задача 2.2. Описание функциональности приложения (диаграмма вариантов использования, user story).	7	06.04.2026	
Задача 2.3. Проектирование модели данных (ER-диаграмма, логическая и физическая схемы БД)	8	10.04.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.4. Разработка макетов страниц (Wireframe)	5	14.04.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка базовой структуры приложения и вёрстка шаблонов страниц	8	21.04.2026	
Задача 3.2. Реализация аутентификации, авторизации и системы ролей (клиент, тренер, администратор)	7	27.04.2026	
Задача 3.3. Разработка CRUD-интерфейса для управления тренировками, расписанием и пользователями	7	03.05.2026	
Задача 3.4. Реализация онлайн-записи на тренировку с проверкой доступных мест и запретом двойной записи	12	09.05.2026	
Задача 3.5. Разработка личного кабинета с историей записей и возможностью отмены	6	15.05.2026	
Задача 3.6. Реализация агрегации данных (статистика посещаемости, загруженность тренеров)	6	20.05.2026	
Задача 3.7. Реализация загрузки файлов (аватар пользователя, фото тренировок)	6	20.05.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	22.03.2026	
Задача 4.2. Деплой приложения на хостинг	4	26.05.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	26.05.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«____» _____ 2026 г. _____ А. С. Кружалов
(подпись)

Дата выдачи задания «02» апреля 2026 г.

Дата сдачи выполненной работы «____» _____ 2026 г.

Задание принял к исполнению

«02» апреля 2026 г. _____ А.И. Минеева
(подпись)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1 Обзор существующих программных продуктов по теме работы	6
1.2 Анализ программных инструментов разработки	7
1.3 Формулировка цели и задач работы	9
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	11
2.1 Анализ целевой аудитории	11
2.2 Описание функциональности приложения	11
2.3 Проектирование модели данных	12
2.4 Разработка макетов страниц	16
ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ	22
3.1 Разработка базовой структуры приложения и вёрстка шаблонов страниц	22
3.2 Реализация аутентификации, авторизации и системы ролей (клиент, тренер, администратор)	24
3.3 Разработка CRUD-интерфейса для управления тренировками, расписанием и пользователями	25
3.4 Реализация онлайн-записи на тренировку с проверкой доступных мест и запретом двойной записи	26
3.5 Разработка личного кабинета с историей записей и возможностью отмены	28
3.6 Реализация агрегации данных (статистика посещаемости, загруженность тренеров)	28
3.7 Реализация загрузки файлов (аватар пользователя, фото тренировок)	29
ЗАКЛЮЧЕНИЕ	31

ВВЕДЕНИЕ

Актуальность темы. В условиях активного развития фитнес-индустрии и роста популярности здорового образа жизни всё большее число людей обращается к услугам фитнес-центров. Управление записью клиентов, расписанием тренировок и взаимодействием с тренерами вручную становится неэффективным и приводит к потере клиентов и снижению качества сервиса. Разработка специализированных веб-приложений, автоматизирующих эти процессы, является актуальной задачей для современного фитнес-бизнеса.

Степень разработанности проблемы. На рынке существуют коммерческие решения для автоматизации фитнес-центров (напр. Mindbody, 1С:Фитнес клуб), однако они ориентированы на крупные сети и обладают избыточным функционалом для небольших студий. Разработка доступного и гибкого веб-приложения с открытым исходным кодом остаётся практически востребованной задачей.

Объект исследования — процессы управления записью клиентов, расписанием тренировок и взаимодействием пользователей в фитнес-центре.

Предмет исследования — программные средства для автоматизации онлайн-записи на тренировки и управления личным кабинетом пользователя в рамках веб-ориентированной платформы.

Целью данной работы является проектирование и реализация веб-приложения для фитнес-центра, обеспечивающего онлайн-запись на тренировки, управление расписанием и личный кабинет пользователя с разграничением прав доступа по ролям.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих программных продуктов по теме работы

На сегодняшний день на рынке представлен ряд программных продуктов для автоматизации управления фитнес-центрами. Для проведения сравнительного анализа были выделены следующие критерии оценки:

- наличие онлайн-записи на тренировки;
- разграничение прав доступа по ролям пользователей;
- наличие личного кабинета клиента;
- возможность загрузки файлов (аватар, фото тренировок);
- стоимость и доступность решения для небольших студий.

Сравнительный анализ существующих решений представлен в табл. 1.1.

Таблица 1.1 – Сравнительный анализ существующих решений

Критерий	Mindbody	1С: Фит- нес клуб	Fitness Pro	Разраб. прило- жение
Онлайн- запись	Да	Нет	Да	Да
Роли поль- зователей	Да	Да	Нет	Да
Личный ка- бинет	Да	Нет	Да	Да
Загрузка файлов	Нет	Нет	Нет	Да
Открытый код	Нет	Нет	Нет	Да

Продолжение таблицы 1.1

Критерий	Mindbody	1С: Фитнес клуб	Fitness Pro	Разраб. приложение
Доступность для небольших студий	Нет	Нет	Да	Да
Модель доступа	SaaS (платная)	Лицензия	Подписка	Бесплатно

Проведённый анализ позволил выявить следующие недостатки существующих решений:

- **Mindbody** [1] — ориентирован на крупные сети, высокая стоимость подписки, избыточный функционал для небольших студий, отсутствует загрузка файлов пользователями.
- **1С:Фитнес клуб** — требует установки и настройки на сервере, отсутствует онлайн-запись и личный кабинет клиента, высокая стоимость лицензии.
- **Fitness Pro** — нет разграничения ролей пользователей, отсутствует загрузка файлов, ограниченная кастомизация.

Таким образом, ни одно из рассмотренных решений не обеспечивает в полной мере потребности небольшого фитнес-центра. Разрабатываемое приложение компенсирует выявленные недостатки: предоставляет онлайн-запись, разграничение прав по ролям, личный кабинет с историей записей и загрузкой файлов при свободном доступе к исходному коду.

1.2 Анализ программных инструментов разработки

Для обоснованного выбора технологического стека был проведён сравнительный анализ инструментов разработки по трём ключевым направлениям: backend, frontend и база данных.

Сравнение backend-фреймворков представлено в табл. 1.2.

Таблица 1.2 – Сравнение backend-фреймворков

Критерий	Go / Gin	Python / Flask	Node.js / Express
Производительность	Высокая	Средняя	Средняя
Типизация	Строгая статическая	Динамическая	Динамическая
Простота деплоя	Один бинарный файл	Требуется Python	Требуется Node.js
Экосистема	Растущая	Богатая	Богатая
Порог входа	Средний	Низкий	Низкий

Для разработки backend выбран язык *Go* с фреймворком *Gin* [2]. *Go* обеспечивает высокую производительность за счёт компиляции в машинный код, строгую статическую типизацию, снижающую количество ошибок, и простой деплой в виде единого бинарного файла без зависимостей.

Сравнение frontend-инструментов представлено в табл. 1.3.

Таблица 1.3 – Сравнение frontend-инструментов

Критерий	React	Vue	Чистый JS
Компонентный подход	Да	Да	Нет
Экосистема	Очень богатая	Богатая	Ограниченная
Популярность	Очень высокая	Высокая	Высокая
Порог входа	Средний	Низкий	Низкий

Для разработки frontend выбран *React* [3] как наиболее распространённая библиотека с богатой экосистемой готовых компонентов и широкой поддержкой сообщества.

Сравнение систем управления базами данных представлено в табл. 1.4.

Таблица 1.4 – Сравнение СУБД

Критерий	PostgreSQL	MySQL	MongoDB
Тип	Реляционная	Реляционная	Документная
Сложные запросы	Отлично	Хорошо	Ограниченно
Агрегация данных	Отлично	Хорошо	Хорошо
Транзакции	Полная поддержка	Полная поддержка	Ограниченно
Открытый код	Да	Частично	Частично

Для хранения данных выбрана СУБД *PostgreSQL* [4]. Реляционная модель данных оптимально подходит для хранения взаимосвязанных сущностей (пользователи, тренировки, записи), а развитые средства агрегации позволяют эффективно формировать статистику посещаемости. Для взаимодействия с базой данных используется ORM-библиотека *GORM* [5], упрощающая написание запросов.

1.3 Формулировка цели и задач работы

Анализ существующих программных продуктов показал, что доступные решения для автоматизации фитнес-центров либо ориентированы на крупные сети и имеют высокую стоимость, либо не обеспечивают полного набора необходимых функций: онлайн-записи, разграничения ролей и загрузки файлов. Анализ инструментов разработки позволил обосновать выбор технологического стека.

Цель работы — проектирование и реализация веб-приложения для фитнес-центра, обеспечивающего онлайн-запись клиентов на тренировки, управление расписанием и личный кабинет пользователя с разграничением прав доступа по ролям.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести анализ существующих программных продуктов и обосновать выбор технологического стека;

2. Спроектировать модель данных и архитектуру приложения;
3. Разработать систему аутентификации, авторизации и управления ролями пользователей (клиент, тренер, администратор);
4. Реализовать CRUD-интерфейс для управления тренировками, расписанием и пользователями;
5. Реализовать онлайн-запись на тренировки с проверкой доступных мест и запретом двойной записи;
6. Разработать личный кабинет пользователя с историей записей и возможностью загрузки файлов;
7. Реализовать агрегацию данных для формирования статистики посещаемости.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории

Разрабатываемое приложение ориентировано на три категории пользователей фитнес-центра. Для каждой роли составлен профиль целевого пользователя (Persona) с описанием потребностей и ожидаемого взаимодействия с системой.

Клиент — физическое лицо, посещающее фитнес-центр. Основные потребности: просматривать актуальное расписание тренировок, записываться на занятия онлайн без необходимости звонить администратору, отслеживать историю своих записей и управлять профилем. Ключевые требования к интерфейсу: простота, быстрый доступ к расписанию, возможность отмены записи.

Тренер — сотрудник фитнес-центра, проводящий тренировки. Основные потребности: управлять собственным расписанием, просматривать список записавшихся клиентов, добавлять и редактировать тренировки, загружать фото тренировок. Требования к интерфейсу: удобный CRUD-интерфейс, наглядное отображение загруженности тренировок.

Администратор — сотрудник, управляющий работой всей платформы. Основные потребности: управлять пользователями и их ролями, просматривать статистику посещаемости, контролировать расписание всех тренеров. Требования к интерфейсу: панель с агрегированными данными, полный доступ к CRUD-операциям.

Таким образом, функциональность приложения определяется совокупностью потребностей трёх ролей, что обуславливает необходимость разграничения прав доступа и разработки отдельных интерфейсов для каждой роли.

2.2 Описание функциональности приложения

Функциональные возможности системы определяются через взаимодействие пользователей с системой. Основные бизнес-кейсы представлены в виде *User Story*:

- «Я, как **клиент**, хочу просмотреть расписание тренировок и записаться на удобное занятие онлайн, чтобы не звонить в фитнес-центр»;

- «Я, как **клиент**, хочу видеть историю своих записей и иметь возможность отменить запись, чтобы управлять своим расписанием»;
- «Я, как **тренер**, хочу добавлять тренировки и видеть список записавшихся клиентов, чтобы планировать рабочий день»;
- «Я, как **администратор**, хочу управлять пользователями и просматривать статистику посещаемости, чтобы оптимизировать работу центра».

На рисунке 2.1 представлена диаграмма вариантов использования (Use Case), отражающая взаимодействие каждой роли с функциями системы.

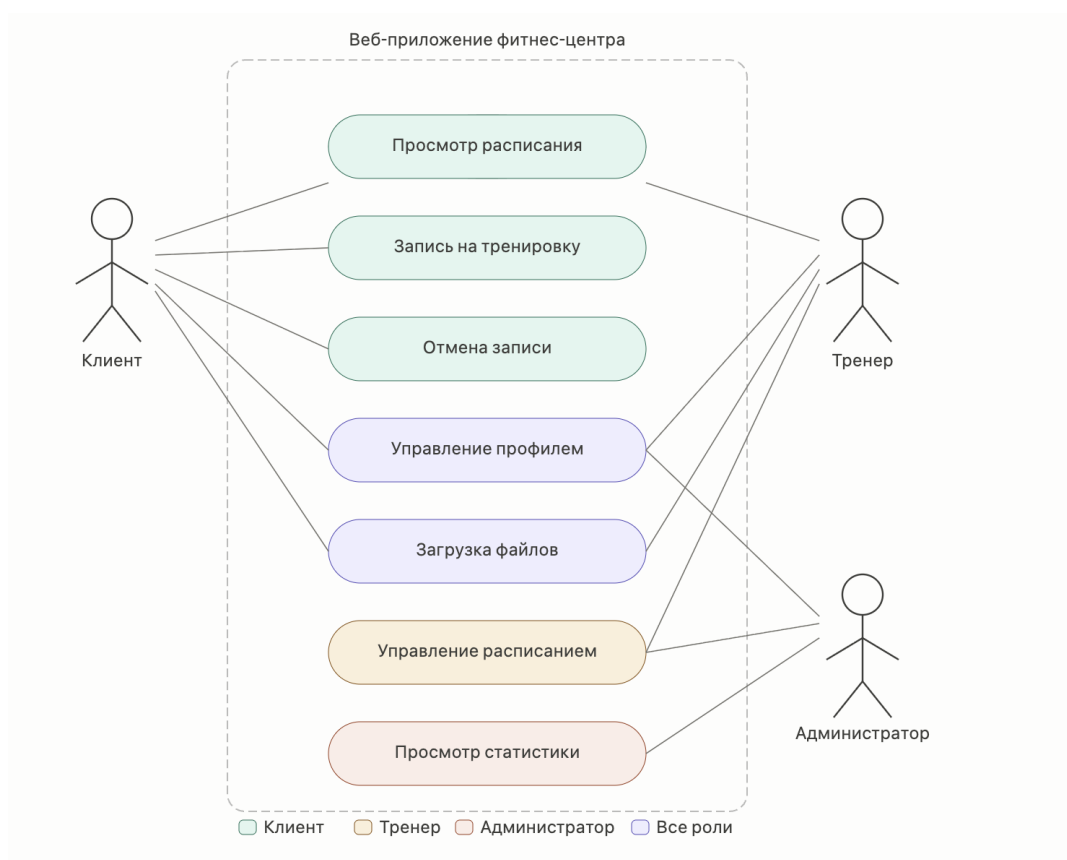


Рисунок 2.1 – Диаграмма вариантов использования системы

2.3 Проектирование модели данных

Модель данных приложения включает три основные сущности: пользователи (users), тренировки (workouts) и записи на тренировки (bookings). Для хранения данных используется СУБД PostgreSQL [4] как наиболее подходящая реляционная система с развитыми средствами агрегации и полной поддержкой транзакций. Взаимодействие с базой данных реализовано через

ORM-библиотеку GORM [5], упрощающую написание запросов на языке Go. ER-диаграмма базы данных представлена на рисунке 2.2.

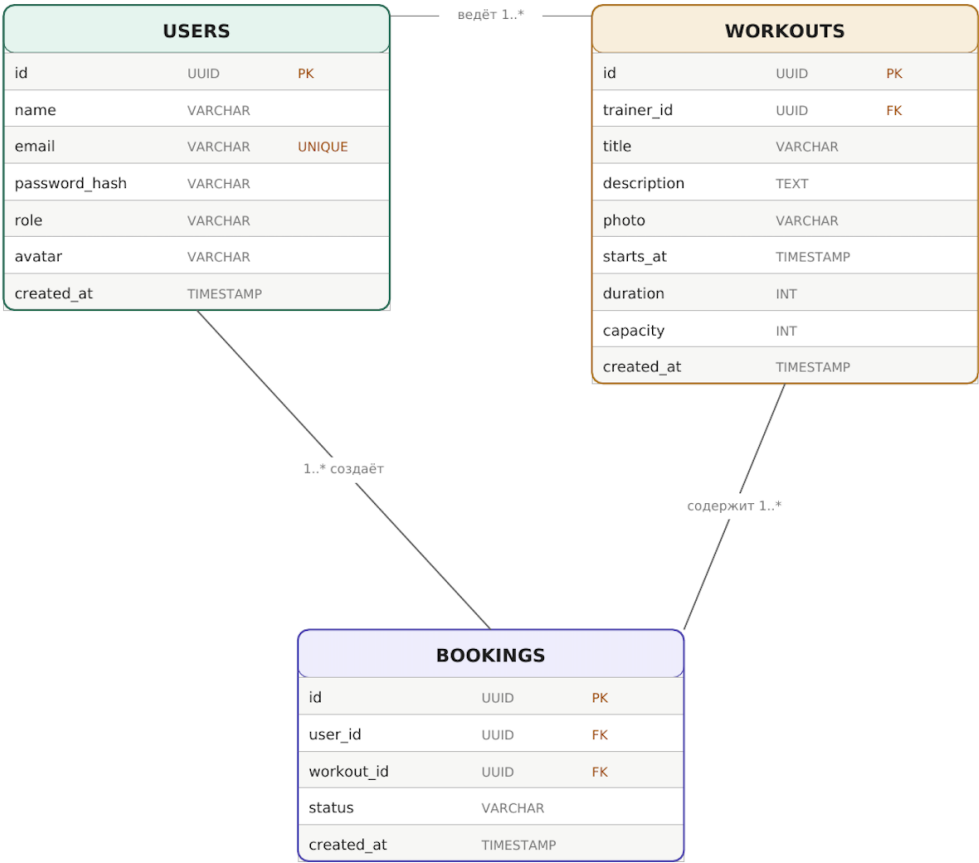


Рисунок 2.2 – ER-диаграмма базы данных

Структура таблицы пользователей представлена в табл. 2.1.

Таблица 2.1 – Структура таблицы пользователей (users)

Поле	Тип	Ограничение	Описание
id	UUID	PRIMARY KEY	Уникальный идентификатор пользователя
name	VARCHAR	NOT NULL	Имя пользователя
email	VARCHAR	UNIQUE, NOT NULL	Электронная почта
password_hash	VARCHAR	NOT NULL	Хэш пароля
role	VARCHAR	NOT NULL	Роль: client, trainer, admin

Продолжение таблицы 2.1

Поле	Тип	Ограничение	Описание
avatar	VARCHAR	NULL	Путь к файлу аватара
created_at	TIMESTAMP	DEFAULT NOW()	Дата регистрации

Структура таблицы тренировок представлена в табл. 2.2.

Таблица 2.2 – Структура таблицы тренировок (workouts)

Поле	Тип	Ограничение	Описание
id	UUID	PRIMARY KEY	Уникальный идентификатор тренировки
trainer_id	UUID	FOREIGN KEY	Ссылка на таблицу пользователей
title	VARCHAR	NOT NULL	Название тренировки
description	TEXT	NULL	Описание тренировки
photo	VARCHAR	NULL	Путь к фото тренировки
starts_at	TIMESTAMP	NOT NULL	Дата и время начала
duration	INT	NOT NULL	Длительность, минут
capacity	INT	NOT NULL	Максимальное число участников
created_at	TIMESTAMP	DEFAULT NOW()	Время создания записи

Структура таблицы записей на тренировки представлена в табл. 2.3.

Таблица 2.3 – Структура таблицы записей (bookings)

Поле	Тип	Ограничение	Описание
id	UUID	PRIMARY KEY	Уникальный идентификатор записи

Продолжение таблицы 2.3

Поле	Тип	Ограничение	Описание
user_id	UUID	FOREIGN KEY	Ссылка на таблицу пользователей
workout_id	UUID	FOREIGN KEY	Ссылка на таблицу тренировок
status	VARCHAR	DEFAULT 'active'	Статус: active, cancelled
created_at	TIMESTAMP	DEFAULT NOW()	Время создания записи

2.4 Разработка макетов страниц

На этапе проектирования были разработаны макеты (Wireframe) ключевых страниц приложения, отражающие общую структуру интерфейса для каждой роли пользователя. Макеты охватывают все основные сценарии взаимодействия: просмотр расписания, аутентификацию, работу в личном кабинете клиента, управление тренировками тренером и администрирование платформы.

Макет страницы расписания (рисунок 2.3) представляет главную публичную страницу приложения, доступную без авторизации. В верхней части расположена навигационная панель с логотипом, ссылками на разделы сайта и кнопками входа и регистрации. Основная область содержит фильтр по дате и тренеру, а также карточки тренировок с фотографией, названием, временем проведения, именем тренера и текущим количеством свободных мест. Каждая карточка содержит кнопку записи, доступную авторизованному клиенту.

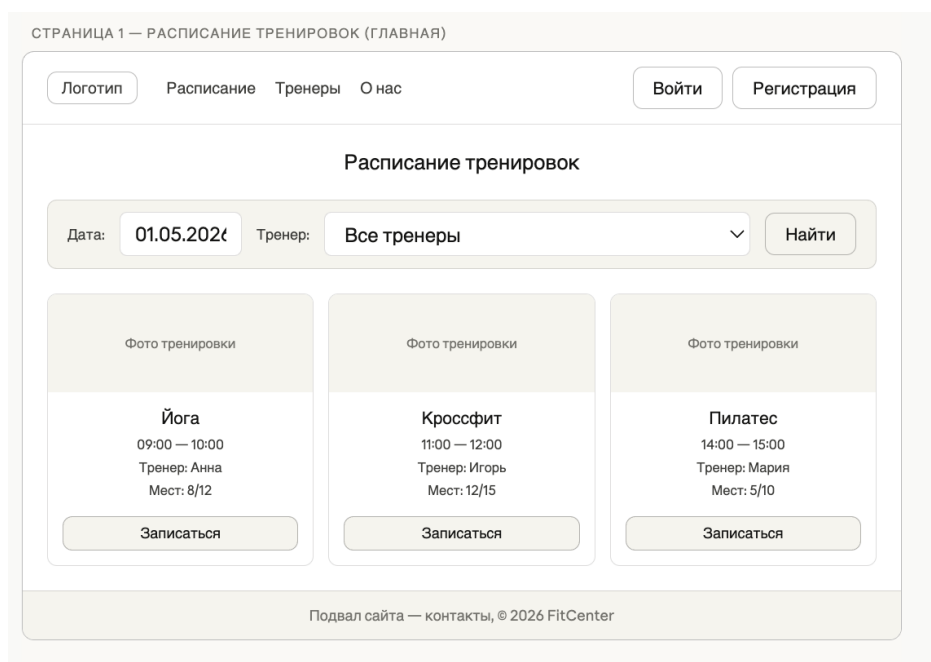


Рисунок 2.3 – Макет страницы расписания тренировок

Макет страницы входа (рисунок 2.4) реализует форму аутентификации, центрированную на странице. Форма содержит поля ввода email и пароля, кнопку подтверждения, ссылку на восстановление пароля и переход к странице регистрации. В верхней части страницы расположен переключатель между вкладками «Войти» и «Регистрация». В верхней части страницы расположен переключатель между вкладками «Войти» и «Регистрация», обеспечивающий навигацию между двумя формами аутентификации.

СТРАНИЦА 4А — ВХОД (ОТДЕЛЬНАЯ ФОРМА)

Логотип Расписание Тренеры О нас

Войти Регистрация

Вход в аккаунт

Email
you@example.com

Пароль
●●●●●●●●

Войти

Забыли пароль? [Восстановить](#)

или

Нет аккаунта? [Зарегистрироваться →](#)

Подвал сайта — © 2026 FitCenter

Рисунок 2.4 – Макет страницы входа в аккаунт

Макет страницы регистрации (рисунок 2.5) содержит форму создания нового аккаунта с полями имени, фамилии, email, пароля, подтверждения пароля и выбора роли (клиент или тренер). Форма также центрирована на странице и использует тот же переключатель вкладок, что и страница входа, что обеспечивает единообразие интерфейса аутентификации.

СТРАНИЦА 4Б — РЕГИСТРАЦИЯ (ОТДЕЛЬНАЯ ФОРМА)

Логотип Расписание Тренеры О нас

Войти Регистрация

Создать аккаунт

Имя
Анастасия

Фамилия
Минеева

Email
you@example.com

Пароль
Минимум 8 символов

Повторите пароль
••••••••

Роль
Клиент

Создать аккаунт

Уже есть аккаунт? [Войти](#) →

Подвал сайта — © 2026 FitCenter

Рисунок 2.5 – Макет страницы регистрации

Макет личного кабинета клиента (рисунок 2.6) разделён на боковую панель и основную рабочую область. В боковой панели отображаются аватар, имя и роль пользователя, а также кнопки навигации: смена фото, мои записи, настройки и выход. Основная область включает сводку статистики (всего записей, предстоящих и посещённых), список активных и прошедших записей с возможностью отмены, а также форму редактирования профиля с полями имени, фамилии, email и телефона.

СТРАНИЦА 2 — ЛИЧНЫЙ КАБИНЕТ (КЛИЕНТ)

Логотип Расписание Тренеры О нас AM Анастасия

Личный кабинет

Фото

Анастасия
Клиент

Изменить фото

Мои записи

Настройки

Выйти

12
Всего записей

3
Предстоящих

9
Посещено

Мои записи

Йога

01.05 09:00

Активна

Отменить

Кроссфит

03.05 11:00

Активна

Отменить

Пилатес

28.04 14:00

Прошла

—

Редактировать профиль

Имя

Анастасия

Фамилия

Минеева

Email

a.mineeva@mail.ru

Телефон

+7 999 123-45-67

Сохранить

Подвал сайта — контакты, © 2026 FitCenter

Рисунок 2.6 – Макет личного кабинета клиента

Макет кабинета тренера (рисунок 2.7) предоставляет интерфейс управления тренировками. Боковая панель содержит навигацию по разделам кабинета: мои тренировки, мои клиенты, и профиль. В основной области отображается список тренировок тренера с указанием даты, времени, заполненности (занятых и общего числа мест) и статуса. Для каждой тренировки доступны кнопки редактирования и удаления. Ниже расположена форма добавления новой тренировки с полями названия, даты и времени, длительности, вместимости, загрузки фотографии и описания.

СТРАНИЦА 5 — КАБИНЕТ ТРЕНЕРА (УПРАВЛЕНИЕ ТРЕНИРОВКАМИ)

Логотип

Тренер

Анна Петрова

Мои тренировки

Мои клиенты

Профиль

Мои тренировки

+ Добавить тренировку

Фильтр по дате

Фото

Йога
01.05.2026 09:00 — 10:00
8 / 12 мест
Открыта

Редакт.

Удалить

Фото

Стретчинг
03.05.2026 10:00 — 11:00
5 / 10 мест
Открыта

Редакт.

Удалить

Фото

Йога (продв.)
05.05.2026 09:00 — 10:30
12 / 12 мест
Мест нет

Редакт.

Удалить

Новая тренировка

Название

Йога

Дата и время

Дд.Мм.Гггг, --:--

Длительность (мин)

60

Мест всего

12

Фото тренировки

Выберите файл

Описание

Краткое описание

Сохранить тренировку

Подвал сайта — © 2026 FitCenter

Рисунок 2.7 – Макет кабинета тренера

Макет панели администратора (рисунок 2.8) включает боковое меню с разделами: дашборд, тренировки, пользователи, расписание и статистика. На дашборде отображаются агрегированные показатели платформы: общее число записей, тренировок, тренеров и клиентов. В разделе управления пользователями представлена таблица со списком всех зарегистрированных пользователей, их email, ролью и датой регистрации. Администратор может добавлять новых пользователей, изменять их роли и удалять учётные записи.

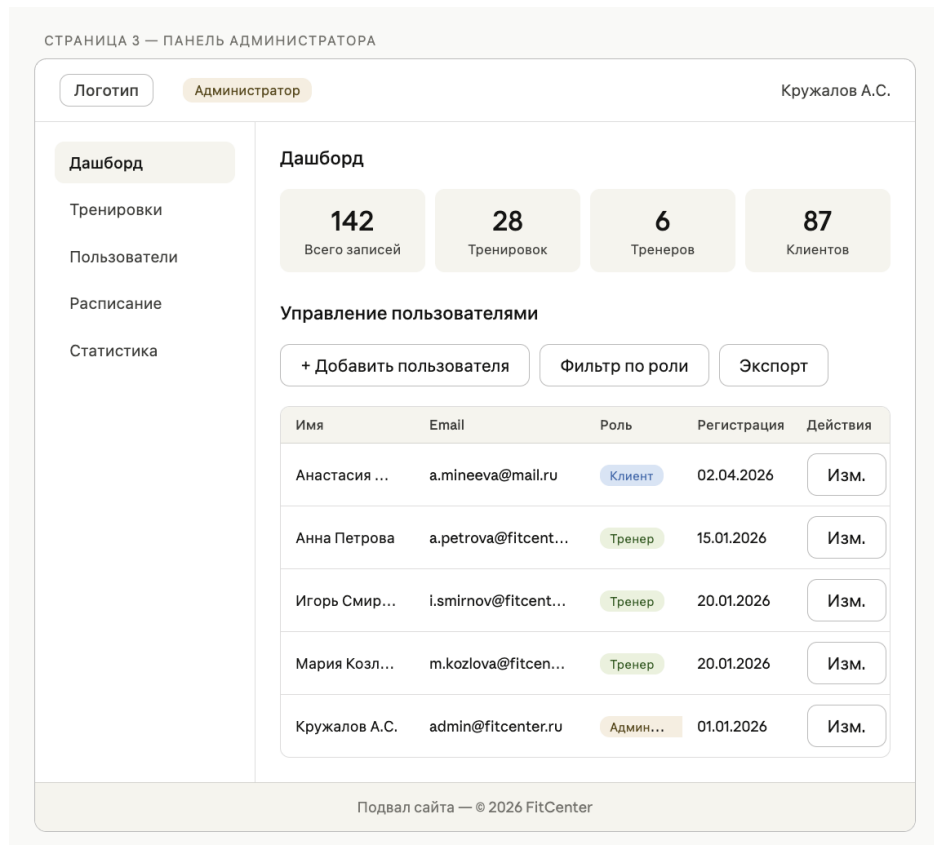


Рисунок 2.8 – Макет панели администратора

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Разработка базовой структуры приложения и вёрстка шаблонов страниц

В соответствии с задачей 3.1, была реализована клиентская часть веб-приложения FitCenter на базе библиотеки **React.js** с использованием маршрутизатора **React Router**. Приложение организовано по компонентному принципу: общие элементы интерфейса вынесены в каталог `components/`, а страницы приложения — в каталог `pages/`. Центральный файл `App.js` определяет маршруты и обеспечивает единую навигационную панель для всех разделов сайта.

Для разграничения доступа к страницам реализован компонент-обёртка `PrivateRoute`, который проверяет наличие авторизованного пользователя и соответствие его роли списку допустимых ролей. Публичными являются страницы расписания (`/`), входа и регистрации (`/login`, `/register`), а также информационные разделы «Тренеры» и «О нас». Личный кабинет клиента доступен по маршруту `/profile`, кабинет тренера — по маршрутам `/trainer` и `/trainer/profile`, панель администратора — по маршрутам `/admin/*`.

Вёрстка страниц выполнена с использованием единого файла стилей `index.css`. Для отображения расписания применяется компонент карточки тренировки, который выводит фотографию, название, дату, время, имя тренера и количество свободных мест. Пример структуры карточки представлен в листинге 1.

Листинг 1 – Фрагмент компонента карточки тренировки (Schedule.js, стр. 8–41)

```
1 function WorkoutCard({ workout, onBook, view }) {
2   const { user } = useAuth();
3   const active = workout.bookings?.filter(b => b.status ===
   'active') || [];
4   const spotsLeft = workout.capacity - active.length;
5   const isFull = spotsLeft <= 0;
6   const bookedCount = Math.min(active.length, workout.
   capacity);
7   const isBooked = active.some(b => String(b.user_id) ===
   String(user?.id));
8   const isPast = new Date(workout.starts_at) < new Date();
9
10  const BookBtn = () => {
11    if (!user) {
12      return (
13        <button
14          className="btn btn-primary btn-full"
15          onClick={() => { window.location.href = '/login';
16        }}
17        >
18        Войти для записи
19      </button>
20    );
21    if (user.role !== 'client') return null;
22    return (
23      <button className="btn btn-primary btn-full"
24        disabled={isFull || isBooked || isPast}
25        onClick={() => onBook(workout.id)}>
26        {isBooked ? ' Вы записаны' : isFull ? Мест' нет' :
27        isPast ? Прошла'' : Записаться''}
28      </button>
29    );
30  };
31 }
```

Серверная часть приложения реализована на языке **Go** с фреймворком **Gin** и организована в виде слоёв: маршруты (transport/), обработчики HTTP-запросов (handlers/), бизнес-логика (service/) и модели данных (models/). Такое разделение упрощает сопровождение кода и соответствует архитектуре, описанной в разделе 2.

3.2 Реализация аутентификации, авторизации и системы ролей (клиент, тренер, администратор)

Согласно задаче 3.2, в системе реализована ролевая модель доступа с тремя ролями: `client` (клиент), `trainer` (тренер) и `admin` (администратор). Для обеспечения безопасности используется протокол **JWT** (JSON Web Token). При успешном входе или регистрации сервер генерирует токен, содержащий идентификатор пользователя (`user_id`) и поле `role`, определяющее права доступа. Пример формирования токена представлен в листинге 2.

Листинг 2 – Генерация JWT-токена при аутентификации (jwtutil.go, стр. 11–18)

```
1 func GenerateToken(user models.User) (string, error) {
2     claims := jwt.MapClaims{
3         "user_id": user.ID.String(),
4         "role":     string(user.Role),
5         "exp":      time.Now().Add(72 * time.Hour).Unix(),
6     }
7     token := jwt.NewWithClaims(jwt.SigningMethodHS256,
8         claims)
9     return token.SignedString([]byte(os.Getenv("JWT_SECRET")))
10 }
```

Пароли пользователей не хранятся в открытом виде: при регистрации и создании учётной записи администратором применяется алгоритм хеширования **bcrypt**. Самостоятельная регистрация в роли тренера запрещена — тренеры создаются только администратором, что предотвращает несанкционированное получение расширенных прав.

На стороне сервера реализованы `middleware`-функции `AuthMiddleware` и `RequireRole`. Первая проверяет наличие и корректность JWT-токена в заголовке `Authorization`, вторая — соответствие роли пользователя требованиям конкретного маршрута. На клиенте контекст `AuthContext` сохраняет токен в `localStorage`, автоматически подставляет его в запросы к API и восстанавливает сессию при перезагрузке страницы.

Маршруты API сгруппированы по уровню доступа (листинг 3): публичные эндпоинты доступны без авторизации, защищённые — после проверки токена, а операции управления тренировками и администрирования — только

для ролей `trainer` и `admin` соответственно.

Листинг 3 – Группировка маршрутов API по ролям (routes.go, стр. 30–43)

```
1    trainer := protected.Group("/")
2    trainer.Use(RequireRole("trainer", "admin"))
3    trainer.POST("/workouts", h.CreateWorkout)
4    trainer.PUT("/workouts/:id", h.UpdateWorkout)
5    trainer.DELETE("/workouts/:id", h.DeleteWorkout)
6    trainer.POST("/workouts/:id/photo", h.UploadWorkoutPhoto
7    )
8
9    adminRoutes := protected.Group("/admin")
10   adminRoutes.Use(RequireRole("admin"))
11   adminRoutes.GET("/users", h.ListUsers)
12   adminRoutes.POST("/users", h.CreateUser)
13   adminRoutes.PUT("/users/:id", h.UpdateUser)
14   adminRoutes.DELETE("/users/:id", h.DeleteUser)
15   adminRoutes.GET("/stats", h.GetStats)
```

3.3 Разработка CRUD-интерфейса для управления тренировками, расписанием и пользователями

В соответствии с задачей 3.3, реализован полный набор операций создания, чтения, обновления и удаления (CRUD) для ключевых сущностей системы.

Управление тренировками доступно тренеру в кабинете `TrainerDashboard` и администратору в разделе «Тренировки» панели `AdminDashboard`. Тренер может создавать, редактировать и удалять только собственные занятия; администратор — тренировки любого тренера. При создании и изменении тренировки проверяется, что дата и время начала не относятся к прошедшему периоду. Данные передаются на сервер через REST API (`POST /api/workouts`, `PUT /api/workouts/:id`, `DELETE /api/workouts/:id`).

Просмотр расписания реализован на странице `Schedule`. Клиент и неавторизованный пользователь могут фильтровать тренировки по дате и тренеру. Список формируется запросом `GET /api/workouts` с параметрами `date` и `trainer_id`. Сервер возвращает тренировки вместе с данными о тренере и активных записях, что позволяет сразу отображать заполненность занятий.

Управление пользователями реализовано в панели администратора. Ад-

министратор просматривает список всех учётных записей, создаёт новых пользователей с заданной ролью, изменяет имя, email и роль, а также удаляет пользователей. При удалении тренера каскадно удаляются связанные тренировки и записи на них, что обеспечивает целостность данных в базе.

3.4 Реализация онлайн-записи на тренировку с проверкой доступных мест и запретом двойной записи

Задача 3.4 реализована в сервисном слое backend-части методом Book, который выполняет запись клиента на выбранную тренировку. Перед созданием записи система последовательно проверяет следующие условия:

- тренировка существует и ещё не началась;
- у клиента нет активной записи на эту же тренировку (запрет двойной записи);
- количество активных записей меньше вместимости (capacity);
- новая тренировка не пересекается по времени с другими активными записями клиента.

Логика проверок представлена в листинге 4.

Листинг 4 – Проверка условий при записи на тренировку (workoutService.go, стр. 194–224)

```
1    var existing models.Booking
2    if err := s.db.Where("user_id = ? AND workout_id = ? AND
   status = 'active'", userID, workoutID).
3        First(&existing).Error; err == nil {
4        return models.Booking{}, ErrAlreadyBooked
5    } else if !errors.Is(err, gorm.ErrRecordNotFound) {
6        return models.Booking{}, fmt.Errorf("check existing
   booking: %w", err)
7    }
8
9    var workout models.Workout
10   if err := s.db.Preload("Bookings", "status = 'active'").
   First(&workout, "id = ?", workoutID).Error; err != nil {
11       if errors.Is(err, gorm.ErrRecordNotFound) {
12           return models.Booking{}, ErrNotFound
13       }
14       return models.Booking{}, fmt.Errorf("find workout: %
   w", err)
15   }
16   if util.WorkoutStarted(workout, time.Now()) {
17       return models.Booking{}, ErrWorkoutPast
18   }
19   if len(workout.Bookings) >= workout.Capacity {
20       return models.Booking{}, ErrNoAvailableSpots
21   }
22
23   var userBookings []models.Booking
24   if err := s.db.Preload("Workout").Where("user_id = ? AND
   status = 'active'", userID).Find(&userBookings).Error;
   err != nil {
25       return models.Booking{}, fmt.Errorf("list user
   bookings: %w", err)
26   }
27   for _, b := range userBookings {
28       if b.WorkoutID != workoutID && util.WorkoutsOverlap(
   workout, b.Workout) {
29           return models.Booking{}, ErrTimeConflict
30       }
31   }
```

На клиентской стороне кнопка «Записаться» на карточке тренировки блокируется, если мест нет, клиент уже записан или тренировка прошла. При

успешной записи отправляется запрос `POST /api/workouts/:id/book`, после чего интерфейс обновляет состояние карточки без перезагрузки страницы. Тексты ошибок («Мест нет», «Вы уже записаны», «Пересечение по времени») локализованы на русский язык в модуле `errors.js`.

3.5 Разработка личного кабинета с историей записей и возможностью отмены

Согласно задаче 3.5, для каждой роли пользователя реализован отдельный личный кабинет, соответствующий макетам, разработанным в разделе 2.4.

Кабинет клиента (`Profile.js`) отображает сводную статистику: общее число записей, количество предстоящих и посещённых тренировок. В разделе «Мои записи» выводится история бронирований с указанием названия тренировки, даты, времени и статуса. Для активных записей на будущие занятия доступна кнопка отмены, которая вызывает эндпоинт `PUT /api/bookings/:id/cancel` и изменяет статус записи на `cancelled`. Клиент также может редактировать имя и загружать аватар.

Кабинет тренера (`TrainerProfile.js`) содержит статистику по проведённым тренировкам и числу записавшихся клиентов, форму редактирования профиля с полями «О себе» и «Опыт», а также загрузку фотографии. Управление расписанием и отметка посещаемости выполняются на странице `TrainerDashboard`.

Кабинет администратора (`AdminProfile.js` и `AdminDashboard.js`) объединяет доступ к статистике платформы, управлению пользователями и тренировками. Получение истории записей клиента реализовано методом `MyBookings`, который загружает связанные тренировки и данные тренера через механизм `Preload ORM`-библиотеки `GORM`.

3.6 Реализация агрегации данных (статистика посещаемости, загруженность тренеров)

В соответствии с задачей 3.6, на сервере реализован эндпоинт `GET /api/admin/stats`, формирующий агрегированные показатели работы фитнес-центра. Метод `GetStats` подсчитывает:

- общее число активных записей (`total_bookings`);
- количество тренировок в системе (`total_workouts`);

- число тренеров (total_trainers) и клиентов (total_clients);
- загруженность каждого тренера — количество его тренировок и записей на них (trainer_load).

Для расчёта загруженности тренеров используется SQL-запрос с группировкой и соединением таблиц users, workouts и bookings (листинг 5). Результаты отображаются на дашборде администратора в виде карточек с ключевыми метриками и таблицы загруженности тренеров.

Листинг 5 – SQL-запрос загруженности тренеров (adminService.go, стр. 159–169)

```

1      if err := s.db.Raw('
2          SELECT u.name as trainer_name,
3              COUNT(DISTINCT w.id) as workouts,
4              COUNT(DISTINCT b.id) as bookings
5      FROM users u
6      LEFT JOIN workouts w ON w.trainer_id = u.id
7      LEFT JOIN bookings b ON b.workout_id = w.id AND b.
      status = 'active'
8      WHERE u.role = 'trainer'
9      GROUP BY u.id, u.name
10     ORDER BY bookings DESC
11     ').Scan(&stats.TrainerLoad).Error; err != nil {

```

Дополнительно тренер в личном кабинете видит количество своих тренировок и уникальных записавшихся клиентов, рассчитанных на стороне клиента на основе полученного списка занятий.

3.7 Реализация загрузки файлов (аватар пользователя, фото тренировок)

Задача 3.7 реализована с разделением статических и динамически загружаемых файлов. Демонстрационные фотографии тренировок размещены в каталоге frontend/public/uploads/workouts/ и отдаются веб-сервером Nginx напрямую при сборке frontend-контейнера. Файлы, загружаемые пользователями в процессе работы (аватары и фото тренировок), сохраняются на диске серверной части в каталоге backend/uploads/ и доступны по URL /uploads/....

Загрузка аватара выполняется запросом POST /api/profile/avatar.

Сервер сохраняет файл в каталог `uploads/avatars/`, генерируя уникальное имя на основе `UUID`, и записывает путь в поле `avatar` таблицы `users`. Загрузка фото тренировки доступна тренеру и администратору через эндпоинт `POST /api/workouts/:id/photo`; файл сохраняется в `uploads/workouts/`, а путь — в поле `photo` таблицы `workouts`.

Вспомогательная функция сохранения файла представлена в листинге 6.

Листинг 6 – Сохранение загруженного файла (upload.go, стр. 11–17)

```
1 func saveUploadedFile(file *multipart.FileHeader, dir string
    , c *gin.Context) (string, error) {
2     filename := uuid.New().String() + filepath.Ext(file.
        Filename)
3     if err := c.SaveUploadedFile(file, dir+"/"+filename);
        err != nil {
4         return "", err
5     }
6     return filename, nil
7 }
```

Для раздачи файлов `backend` регистрирует статический маршрут `r.Static("/up`. В `production`-окружении `Nginx` сначала ищет запрошенный файл среди статических ресурсов `frontend`, а при отсутствии — проксирует запрос на `backend`, что обеспечивает корректную работу как демонстрационных, так и пользовательских изображений. Каталог `backend/uploads` монтируется как `Docker-volume`, поэтому загруженные файлы сохраняются при перезапуске контейнеров.

Таким образом, в третьей главе описана практическая реализация всех задач раздела 3 плана работ: от базовой структуры интерфейса до загрузки файлов и формирования статистики. Разработанное приложение `FitCenter` развёртывается с помощью **Docker Compose** и включает три сервиса: `frontend` (`Nginx` + `React`), `backend` (`Go` + `Gin`) и СУБД `PostgreSQL`.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсового проекта разработано веб-приложение FitCenter для автоматизации деятельности фитнес-центра, обеспечивающее онлайн-запись клиентов на тренировки, управление расписанием и личный кабинет пользователя с разграничением прав доступа по ролям. Все семь задач раздела разработки, определённых в техническом задании, выполнены в установленные сроки.

На основе результатов проектирования и реализации можно сформулировать следующие **выводы**:

1. Сравнительный анализ существующих решений (Mindbody, 1С:Фитнес клуб, Fitness Pro) показал, что ни одно из них не удовлетворяет одновременно требованиям доступности для небольших студий, наличия ролевой модели и загрузки файлов. Разработанное приложение компенсирует все выявленные недостатки при полностью открытом исходном коде.
2. Выбранный технологический стек (React, Go/Gin, PostgreSQL, Docker Compose) обеспечил чёткое разделение ответственности между уровнями представления, бизнес-логики и хранения данных, что подтвердилось в ходе реализации: добавление новых проверок при бронировании и расширение статистики администратора не потребовали изменений в других слоях приложения.
3. Реализация бизнес-логики записи на тренировку — с проверкой вместимости, запретом двойной записи и выявлением пересечений по времени — полностью выполнена на стороне сервера, что исключает некорректное состояние данных вне зависимости от поведения клиентской части.

Рекомендации и предложения. Для практического внедрения приложения в деятельность реального фитнес-центра рекомендуется перенести секреты конфигурации (JWT-ключ, пароль базы данных) из файла `docker-compose.yml` в переменные окружения, а также добавить валидацию типа и размера загружаемых файлов на стороне сервера. На уровне инфраструктуры целесообразно использовать модель облачного развёртывания с настройкой CI/CD-пайплайна (например, GitHub Actions), что обеспечит автоматическую сборку и доставку

обновлений без ручного вмешательства. Для повышения отказоустойчивости в условиях реальной нагрузки рекомендуется вынести хранилище загружаемых файлов из локальной файловой системы в объектное хранилище (S3-совместимое), а соединение с базой данных — защитить SSL-шифрованием.

Перспективы развития. В качестве направлений дальнейшего развития системы целесообразно рассмотреть несколько уровней. На функциональном уровне — интеграцию модуля онлайн-оплаты абонементов и разовых посещений, реализацию push-уведомлений о предстоящих тренировках и напоминаний об отмене записи, а также разработку мобильного клиента на основе существующего REST API, который уже содержит все необходимые эндпоинты. На аналитическом уровне перспективным направлением является внедрение модуля предиктивной аналитики: на основе накопленной истории посещаемости возможно прогнозирование спроса на конкретные виды тренировок и автоматическое формирование рекомендаций по оптимизации расписания. На архитектурном уровне, при значительном росте числа пользователей, целесообразен переход от монолитного развёртывания к микросервисной архитектуре с выделением сервисов аутентификации, бронирования и уведомлений в самостоятельные компоненты.

При разработке пояснительной записки к курсовому проекту строго соблюдались требования [?], определяющие структуру и правила оформления научно-исследовательских работ.

СПИСОК ЛИТЕРАТУРЫ

- [1] Mindbody: Business Management Software for Wellness [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://www.mindbodyonline.com> (дата обращения: 20.03.2026).
- [2] Gin Web Framework Documentation [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://gin-gonic.com/docs/> (дата обращения: 01.04.2026).
- [3] React Documentation: Quick Start [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://react.dev> (дата обращения: 27.03.2026).
- [4] PostgreSQL Documentation [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://www.postgresql.org/docs/> (дата обращения: 01.04.2026).
- [5] GORM: The fantastic ORM library for Golang [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://gorm.io/docs/> (дата обращения: 01.04.2026).